Ders 19 Çalışma Özeti

Ders 19 Çalışma Özeti

Genel Konular

- Bayramdan sonraki ilk ders. Konular:
  - Fonksiyon çağrısının arka planı: stack bellek, program counter'ın yedeklenmesi ve fonksiyon kullanmanın maliyeti.
  - Fonksiyonlar için kavramlar: overloading, fonksiyon/metot/prosedür/subroutine.
  - En yakın çift (closest pair) problemi: tek boyutta, iki boyutta ve k boyutta.
  - Çöpçatan ve doktor-teşhis senaryoları.
  - Özyinelemeli (recursive) fonksiyonlara giriş.
- Önceki dersin tekrarı
  - Dizide en küçük veya en büyük elemanı bulan fonksiyon değeri `return` ile, indisi pointer ile döndürür. C'de ve Java'da gerçek anlamda call by reference yoktur, C++ ve C#'ta vardır. C'de pointer ile benzeri yapılır.
  - Matrisin en büyüğünü bulmak için aynı fonksiyon satır satır çağrılmıştı. Bir kez yazıp birçok kez çağırmak fonksiyonların temel avantajıdır. Programlama dillerinin hazır fonksiyonları da bu yüzden vardır.
- Fonksiyon çağrısının adımları (`a = fakt(5);` örneği)
  1. Parantez içindeki ifadeler hesaplanır (ör. `fakt(5 + 5)` için önce toplama).
  2. Parametreler fonksiyonun değişkenlerine kopyalanır (birden fazla parametre varsa sırayla).
  3. Fonksiyonun gövdesi `return`'e kadar çalışır.
  4. `return` edilen değer çağrının yerine düşer.
  5. Atama yapılır: değer `a`'ya kopyalanır.
  - `void` fonksiyonda dördüncü adımda değer dönmez, fonksiyon sadece tamamlanır ve program devam eder.
- Fonksiyon dönüşünde nereden devam edileceği nasıl bilinir?
  - `main` `f`'i, `f` `g`'yi, `g` `h`'yi çağırıyorsa her çağrıdan sonra programın kaldığı satıra dönülmesi gerekir. Bu satırlar bir yerde saklanmalıdır.
  - Tek bir değişkende saklanırsa iç içe çağrıda önceki değer kaybolur (6 yerine 105 yazılınca `main`'e dönülemez).
  - Çözüm: ilk giren son çıkar (LIFO) mantığıyla çalışan stack veri yapısı. Çağrıda program counter'ın değeri stack'e eklenir. `return` görülünce en üstteki değer çekilir ve program counter'a yazılır.
  - Stack'e yalnızca dönüş adresi değil, fonksiyonun yerel değişkenleri ve işlemcinin durum (flag) yazmaçları da kaydedilir. Fonksiyon bitince geri yüklenir. Ayrıntıları Computer Organization dersinde görülür.
  - Bu alan, işletim sisteminin prosese ayırdığı bellek içindeki stack bölgesidir.
- Fonksiyon kullanmanın dezavantajları
  - Her çağrıda yedekleme ve geri yükleme yapıldığı için zaman kaybı vardır. Bu kayıp nanosaniyeler mertebesindedir. Modülerliğin getirdiği fayda daha önemlidir. Nesneye yönelik dillerde tasarım tamamen bu yapı üzerine kuruludur.
  - Stack bölgesi sınırlıdır. Heap belli bir sınıra kadar büyüyebilir, stack büyüyemez. Çok fazla iç içe çağrı stack overflow hatasına yol açar.
- Akış diyagramında fonksiyon ve isimlendirme
  - Fonksiyon tanımında başlangıç sembolüne fonksiyon adı ve giriş parametreleri, çıkışına dönüş değerleri yazılır. Teorik olarak birden fazla çıkış olabilir. C'de bu pointer'larla yapılır.

  - Fonksiyon çağrısı işlem kutusu (dikdörtgen) içinde gösterilir. Örnek: `maxD, maxIndis = MaxBul(A, N)`.
  - Overloading: aynı adla, parametre sayısı, sırası veya tipi farklı birden fazla fonksiyon tanımlamak. Derleyici çağrıdaki parametrelere göre doğru fonksiyonu seçer. C'de yoktur, Java gibi nesneye yönelik dillerde vardır.
  - Aynı kavramın farklı adları: function (C), method (Java, C# gibi nesneye yönelik diller), procedure (Pascal), subroutine (genel kavram). Hepsi bir modüldür.
- En yakın çift (closest pair) problemi
  - Öneri sistemlerinin (ör. Spotify'ın haftalık keşif listesi) temelinde yatan problemdir. Hoca, müziklerin akustik özelliklerinden en benzer şarkıların önerilmesini örnek olarak gösterir. Büyük bir müzik veri setinden de söz eder.
  - "Bana arkadaşını söyle, sana kim olduğunu söyleyeyim": birine en yakın olan, ona en çok benzeyendir.
  - Tek boyut: sayı doğrusundaki noktalar (3, 4, 8, 7, 5, 1 gibi). Her nokta diğerleriyle karşılaştırılır, uzaklık mutlak farktır. En yakın çiftler 3-4, 4-5 ve 7-8 (uzaklık 1).
  - Benzetme örneği: "dayıma benzerim". Yüz şekli veya kararlılık gibi her özellik bir sayı ekseni olarak düşünülebilir. En yakın kişi en çok benzenen kişidir.
  - Akış diyagramı (`EnYakin(A, n)`, çıktı `indis1`, `indis2`): `minDist` başlangıçta sonsuz, `i` 0'dan `n-2`'ye, `j` `i+1`'den `n-1`'e. `dist = |A[i] - A[j]|`. `dist < minDist` ise `minDist`, `indis1 = i`, `indis2 = j` güncellenir. `j` `i+1`'den başlar, çünkü bir noktayı kendisiyle karşılaştırmaya ve aynı çifte iki kez bakmaya gerek yoktur.
  - İki indis döndürmek için biri `return`, diğeri pointer ile ya da ikisi de pointer ile döndürülebilir.
  - Hoca, bu işin n^2'den daha iyi, $n \log n$'de yapılabildiğini söyler.
- İki ve k boyutlu uzaklık
  - Her nokta için iki özellik varsa veri `n × 2`'lik bir matriste tutulur (satırlar noktalar, sütunlar özellikler). k özellik için `n × k`'lık matris kullanılır.
  - Manhattan uzaklığı (L1 normu): $|x_i - x_j| + |y_i - y_j|$.
  - Öklid uzaklığı: $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.
  - Eğim veya türev uzaklığı göstermez, yalnızca artış veya azalışı gösterir.
  - k boyutta üçüncü bir döngü (`l`, 0'dan `k-1`'e) gerekir: `dist += |A[i][l] - A[j][l]|`. Her çift için `dist` önce sıfırlanmalıdır. Öklid için kareler toplanıp sonunda karekök alınır.
  - Bir öğrenci, yalnızca en yakını bulmak için karekök almaya gerek olmadığını söyledi. Hoca bunu doğrular: karekök sıralamayı değiştirmez.
  - İkinci, üçüncü en yakın çiftler de istenirse bulunan çiftler ve uzaklıkları bir matriste tutulur. Yeni bir uzaklık daha küçükse (büyük olasılıkla insertion sort mantığıyla) yerine yerleştirilir ve büyükler kaydırılır.
  - Özelliklerin ölçekleri farklıysa (yaş 20-70, maaş 1000-20000) büyük ölçekli özellik farkı bastırır. Bu durumda özellikler normalize edilir. Bu kısım istatistik ve veri madenciliği konusudur.
- Çöpçatan senaryosu (`Copcatan.c`)
  - `ERKEK` kişilik ve `KADIN` kişilik iki ayrı matris. Herkese `SORU` tane soru sorulur, cevaplar 1-5 arasında. Matrisler `matris_doldur` fonksiyonuyla `1 + rand() % 5` ile rastgele doldurulur. `srand(time(NULL))` `main`'in başında bir kez çağrılır.
  - Değişkenleri alt alta yazıp yanına açıklama eklemek kodun okunabilirliğini artırır.
  - `void enUygunBul(int E[][SORU], int N, int K[][SORU], int M, int *i1, int *i2)`: `min_d = INT_MAX` (`<limits.h>`). `i` erkekler, `j` kadınlar, `k` sorular üzerinde döner: `dist += abs(E[i][k] - K[j][k])`. Her çiftten sonra `dist` sıfırlanır. En küçük uzaklıkta `*i1 = i; *i2 = j;`.
  - İki farklı matris olduğu için `j` `i+1`'den değil 0'dan başlar. Erkek-kadın ve kadın-erkek uzaklığı simetriktir, döngülerin sırası önemli değildir.
  - Fonksiyona aynı adlı hem matris (`M`) hem tam sayı (`m`) parametresi verilmesi sorun çıkardı. Parametre adları farklı olmalıdır.
  - Hoca sonucu elle kontrol etti: en küçük uzaklıklı çift (uzaklık 2) doğru bulunmuştu.
- Kategorik cevaplar
  - Cevaplar sayısal değil kategorikse (1-5 arası şıklar, aralarındaki fark anlamsız) uzaklık farklı hesaplanır: aynı cevaplar 0, farklı cevaplar 1 sayılır. Yani uyuşmazlıklar sayılır. Benzerlik ve uzaklık birbirinin tersidir.

  - `int findDistance(int x, int y, char c)`: `c == 'c'` ise `if (x == y) return 0; else return 1;`, değilse `return abs(x - y);`. `enUygunBul2` fonksiyonu bu ek `char` parametresini alır.
  - Fonksiyonun ortasında `return` kullanılabilir. `return` çalışınca fonksiyonun geri kalanı çalışmaz.
  - Bu fonksiyonlar ayrı bir C dosyasına yazılıp başlık dosyalarıyla başka programlarda kullanılabilir. Bu gelecek dönemin konusudur.
- Doktor-teşhis senaryosu (`Doktorlar.c`)
  - `DOKTOR` sayıda doktorun `HASTA` sayıda hastaya koyduğu teşhisler (1-100 arası hastalık kodları) `D × H`'lik bir matriste, doğru teşhisler `H` elemanlı bir dizide tutulur. Böyle bir problemde veri yapısının matris olduğu hemen görülmelidir.
  - En başarılı doktor: önce her doktor için doğru teşhis sayısı bir `temp` dizisine yazılır (`if (D[i][j] == dogru[j]) t++;`). Sonra bu dizinin en büyük elemanının indisi önceden yazılan en büyüğü bulma fonksiyonuyla bulunur.
  - Bir öğrenci sıralamanın daha iyi olup olmayacağını sordu. Hoca, sıralamanın en az n^2 (basit algoritmalarla) gerektirdiğini, en büyüğü bulmanın ise tek döngüde n adımda yapıldığını söyler. Sınavda sıralama yapılırsa puan çok düşer.
  - Çalışma soruları: birbirine en çok benzeyen iki doktoru bulun (kategorik uzaklık, tek matris, `j = i + 1`). En az konulan teşhisi bulun. Hasta teşhisleri çoğunluk oyuyla seçilseydi kimlere yanlış teşhis konurdu?
- Özyinelemeli (recursive) fonksiyonlar
  - Bir fonksiyon kendini doğrudan veya başka bir fonksiyon üzerinden dolaylı olarak çağırıyorsa özyinelemelidir. Bir fonksiyonun başka bir fonksiyonu çağırması tek başına özyineleme değildir.
  - Deneme: `void f() { printf("deneme"); f(); }`. Durma koşulu olmadığı için stack dolar ve işletim sistemi programı sonlandırır. `static int x` ile çağrı sayısı sayıldı: 64905. `static` değişken bir kez ilk değer alır, her çağrıda sıfırlanmaz.
  - `for` içinde `f()` 100.000 kez çağrılırsa sorun olmaz. Her çağrı bitince stack'ten çıkar, stack dolup boşalır, birikmez.
  - Neden özyineleme? Bir problemin çözümü kendinden küçük alt problemlerin çözümüne bağlıysa kullanılır. $5! = 5 \cdot 4!$. $4!$ biliniyorsa tek çarpma yeter.
  - İki aşama: özyineleme bağıntısını (recurrence relation) yazmak ve durma koşulunu (base case) belirlemek.
  - Faktöriyel: $f(n) = n \cdot f(n-1)$ ($n \geq 2$), $f(n) = 1$ ($n = 0$ veya 1). Durma koşulu olmadan $f(0) = 0 \cdot f(-1)$ diye devam edilir, hem sonuç yanlış olur hem stack taşar. Derste durma koşulsuz `fakt(3)` yine 64905 çağrıda durdu.
  - Kod: `int fakt(int n) { if (n >= 0 && n < 2) return 1; return n * fakt(n - 1); }`. `if return` ettiği için `else` yazmaya gerek yoktur. $0! = 1$ olduğu için koşul `n >= 0` ile yazıldı.
  - Hızlı üs alma: $x^n = x^{n/2} \cdot x^{n/2}$. $x^{n/2}$ biliniyorsa tek çarpma yeter. Böylece $n-1$ çarpma yerine yaklaşık $\log_2 n$ çarpma yapılır: x^{1024} için 1023 yerine 10 çarpma (hoca 1024 kez dedi; 1'den başlanmazsa doğrusu 1023'tür). Analizine Cuma günü devam edilecek.

Hocanın Özellikle Vurguladığı Kısımlar

- Fonksiyon çağrıları stack'te yedeklenir. Bu küçük bir zaman maliyeti getirir ve stack sınırlıdır.
- En yakın çift probleminde `j i + 1`'den başlar. Her çift için uzaklık değişkenini sıfırlayın.
- En büyüğü bulmak yeterliyse sıralama yapmayın.
- Özyinelemeli fonksiyonda durma koşulunu mutlaka yazın.
- Proje için şimdiden algoritma araştırmaya başlayın.

Kısa Tekrar Notları

- Çağrı adımları: argüman hesabı, kopyalama, gövde, `return`, atama.
- Stack: LIFO. Dönüş adresi, yerel değişkenler ve yazmaçlar saklanır.
- Overloading C'de yoktur.
- Manhattan: $\sum |a_l - b_l|$. Öklid: $\sqrt{\sum (a_l - b_l)^2}$. Kategorik: farklı cevap sayısı.

- `INT_MAX` `<limits.h>`'te tanımlıdır.
- Özyineleme = bağıntı + durma koşulu. `fakt(n) = n * fakt(n - 1)`, `fakt(0) = fakt(1) = 1`.

Detaylı Açıklamalar

Çöpçatan, doktor ve faktöriyel kodları

Derste yazılan fonksiyonlar sabit test verileriyle bir programda toplanıp `gcc -Wall -std=c89 -pedantic` ile denendi:

```
int findDistance(int x, int y, char c){
    if (c == 'c') {
        if (x == y) return 0;
        return 1;
    }
    return abs(x - y);           /* abs: <stdlib.h> */
}
void enUygunBul2(int E[][SORU], int N, int K[][SORU], int M, char c, int *i1, int *i2){
    int i, j, k, min_d = INT_MAX, dist;
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            dist = 0;
            for (k = 0; k < SORU; k++) dist += findDistance(E[i][k], K[j][k], c);
            if (dist < min_d) { min_d = dist; *i1 = i; *i2 = j; }
        }
}
int fakt(int n){
    if (n >= 0 && n < 2) return 1;
    return n * fakt(n - 1);
}
```

- Erkekler (4 5 2), (1 3 3) ve kadınlar (2 4 5), (4 3 2), (1 1 3) için sayısal uzaklıkta 0. erkek ile 1. kadın bulundu (uzaklık 2). Kategorik uzaklıkta da aynı çift bulundu (tek uyuşmazlık).
- 4 doktorlu, 5 hastalı bir test matrisinde en iyi doktor 0 (5 doğru teşhis) bulundu.
- `fakt(3)` 6, `fakt(0)` 1, `fakt(5)` 120 verdi.

Ders sonrası notlar

- **`abs` ve `INT_MAX` başlık dosyaları:** Derste `abs` için `<math.h>` eklendi. `INT_MAX` için altyazıda "list.h" geçiyor (01:41:43), büyük olasılıkla `limits.h` söylendi. `abs` (tam sayı mutlak değeri) `<stdlib.h>`'te, `INT_MAX` `<limits.h>`'te tanımlıdır. `<math.h>`'teki karşılığı `fabs`'tır ve `double` ile çalışır.
- **Derste görülen yazım hatası:** Ekrandaki kodda `if (dist < min_d)` bloğunda büyük olasılıkla `mid_d = dist;` yazılmıştı ve iç döngüde `K[j][j]` vardı. `K[j][j]` derste `K[j][k]` olarak düzeltildi. `min_d` güncellenmezse her zaman `INT_MAX` ile karşılaştırılır ve en yakın değil, son karşılaştırılan çift bulunur.
- **`return` ve `else`:** Hoca, `if` kolunda `return` yazıldıysa `else` kolu için de `return` yazılması gerektiğini, yoksa derleme hatası alınacağını söyledi. C'de `void` olmayan bir fonksiyonun sonunda `return` eksikse derleyici en fazla uyarı verir (`-Wall` ile "control reaches end of non-void function"). Derleme yine başarılı olur. Ama dönen değer kullanılırsa davranış tanımsızdır. Bu yüzden her yolun bir `return` ile bitmesi gerekir.
- **Karşılaştırma sayısı:** Tek boyutlu en yakın çift için hoca $\frac{n(n+1)}{2}$ dedi. `j i + 1`'den başladığında karşılaştırma sayısı $\frac{n(n-1)}{2}$'dir. Tek boyutta dizi önce sıralanırsa ($n \log n$) en yakın çift mutlaka yan yana iki eleman olur ve tek bir taramayla bulunur.
- **Stack pointer:** Derste stack pointer'ın 8 bitlik olduğu söylendi. Stack pointer, işlemcinin adres genişliğinde bir yazmaçtır (32 bit işlemcide 32, x86-64'te 64 bit). Stack belleğin kendisi RAM'dedir, stack pointer onun tepesini gösterir.

- **64905 çağrı sınırı:** Hoca bu sayıyı bilgisayarın RAM miktarına bağladı. Asıl belirleyici, programa ayrılan stack boyutudur. Windows'ta varsayılan değer 1 MB'tır ve RAM'e bağlı değildir. Ders sonrasında Linux'ta (8 MB stack) aynı deneme 500.000'den fazla çağrıya ulaştı.
- **Negatif sayıyla faktöriyel:** Derste yazılan `fakt`'ta `n` negatifse `if` koşulu sağlanmaz ve `n * fakt(n - 1)` sonsuza kadar devam eder (stack overflow). Ya koşul `if (n < 2) return 1;` olarak yazılmalı ya da negatif girişler fonksiyon çağrılmadan önce engellenmelidir.
- **Büyük asal sayılar ve şifreleme:** Hoca, büyük bir asal sayıya sahip olmanın daha küçük şifreleri çözmeyi kolaylaştırdığını söyledi. RSA gibi şifrelemelerin güvenliği, iki büyük asal sayının çarpımını çarpanlarına ayırmanın zorluğuna dayanır. Daha büyük bir asal sayıyı bilmek, bu çarpımı ayırmaya yardım etmez.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.